

Exploring and Testing APIs with Postman

1. Api choice and base url

The API selected for this assignment is DummyJSON (<https://dummyjson.com>), a free, publicly hosted REST API which also acts as a MOCK API too that requires no signup or API key, and provides genuine JWT-based authentication alongside realistic CRUD-capable resources (products, users, carts, posts, and more).

- **Base URL:** <https://dummyjson.com>

This API was chosen over alternatives such as JSONPlaceholder because JSONPlaceholder has no authentication layer at all. DummyJSON supports the full assignment brief — login, token issuance, and CRUD — without any account setup.

- **Resource used for CRUD demonstration:** Products (</products>).

2. Authentication Method

DummyJSON uses stateless JWT (JSON Web Token) authentication, functionally equivalent to an OAuth2 password-grant flow:

- A POST request to </auth/login> with a username and password returns an accessToken (JWT) and a refreshToken.
- The accessToken is sent as a Bearer token in the Authorization header on every subsequent protected request.
- When the access token nears expiry, POST </auth/refresh> exchanges the refreshToken for a new accessToken without re-sending credentials.

Sample Login Request

```
POST https://dummyjson.com/auth/login
Content-Type: application/json
```

```
{
  "username": "{{username}}",
  "password": "{{password}}",
  "expiresInMins": 30
}
```

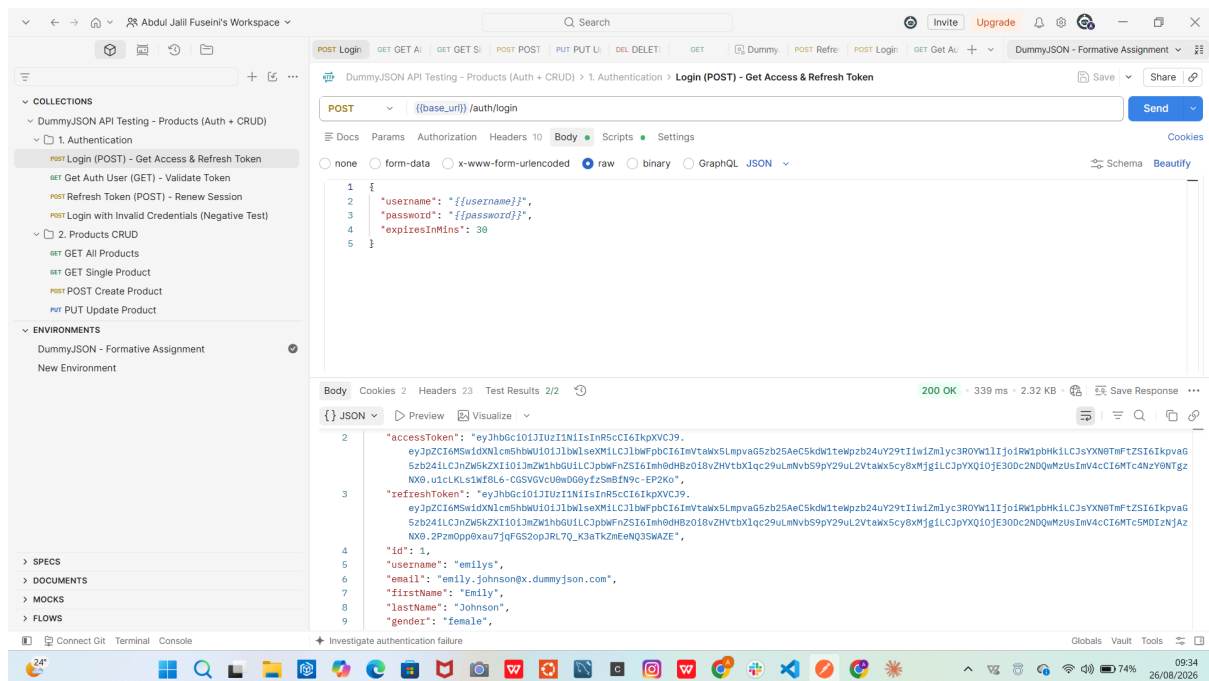
In the postman environment, the actual value of username is Emilys and that of password of emilypass which will be swapped in during the request.

Sample Login Response (200 OK)

```
{
  "id": 1,
  "username": "emilys",
  "email": "emily.johnson@x.dummyjson.com",
  "firstName": "Emily",
  "lastName": "Johnson",
  "accessToken": "eyJhbGciOiJIUzI1NiIs... ",
  "refreshToken": "eyJhbGciOiJIUzI1NiIs..."
}
```

The accessToken is immediately captured by a Postman test script and stored in the environment variable auth_token, so it never has to be copy-pasted manually.

Screenshot of the request and the associated response



3. API Endpoint Documentation

The table below documents every endpoint exercised in the collection, matching the brief's requirement to record URL, method, and auth requirement for each.

Method	Endpoint	Purpose	Auth Required
POST	/auth/login	Log in and receive accessToken + refreshToken	No (this is the login). No bearer token yet.

GET	/auth/me	Return the profile of the currently authenticated user	Yes (Bearer token)
POST	/auth/refresh	Issue a new access token using the refresh token	No (uses refreshToken)
GET	/products	Retrieve a paginated list of all products	Yes (Bearer token) but still works without it.
GET	/products/{id}	Retrieve a single product by ID	Yes (Bearer token) but still works without it
POST	/products/add	Create (simulate) a new product	Yes (Bearer token) but still works without it
PUT	/products/{id}	Update (simulate) an existing product	Yes (Bearer token) but still works without it
DELETE	/products/{id}	Delete (simulate) a product	Yes (Bearer token) but still works without it

The required Headers and Authorization documented

- Required headers – Content-Type: application/json is used on every request with a body such as POST and PUT. But for all get and Delete request which doesn't have a request body, it is not required.
- Authorization: Bearer {{auth_token}} is used on every protected request. Even though it will still work without it. But to simulate the actual application in real production API's, such protected request will need some sort of authorization.

4. CRUD(CREATE,READ,UPDATE,DELETE) Operations

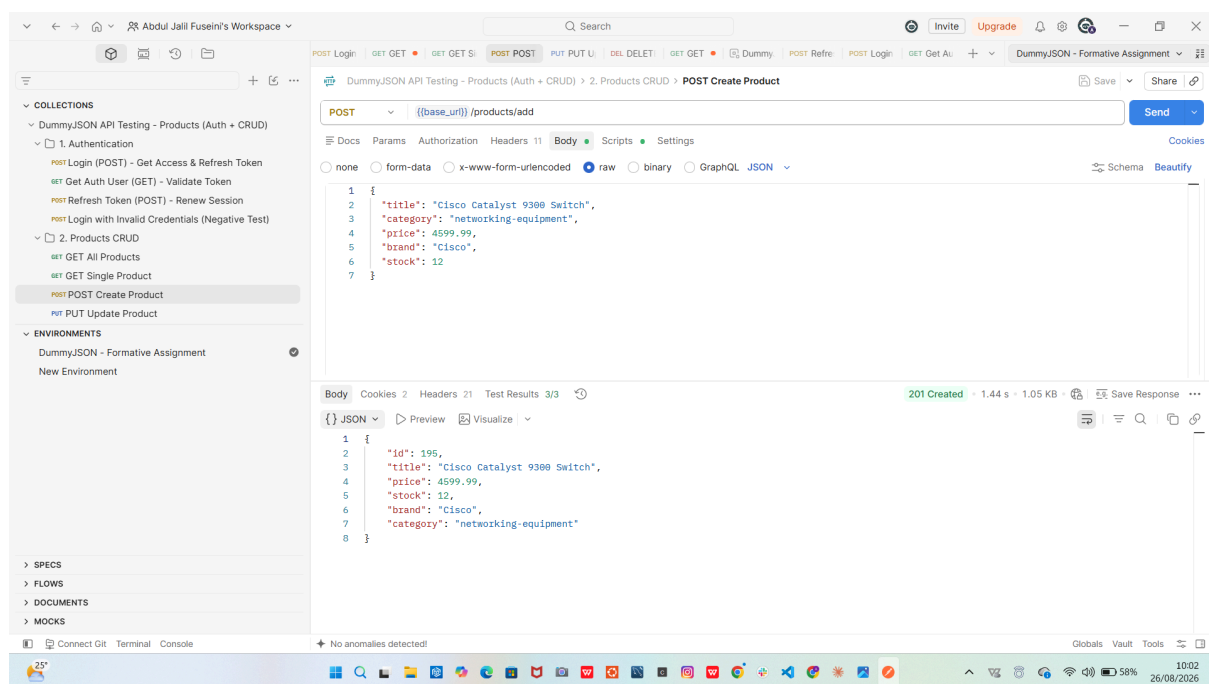
All four CRUD operations were demonstrated against the Products resource, using the token obtained in the login stage.

- **CREATE — POST /products/add (creating a new product)**

```
POST https://dummyjson.com/products/add
Authorization: Bearer {{auth_token}}
Content-Type: application/json

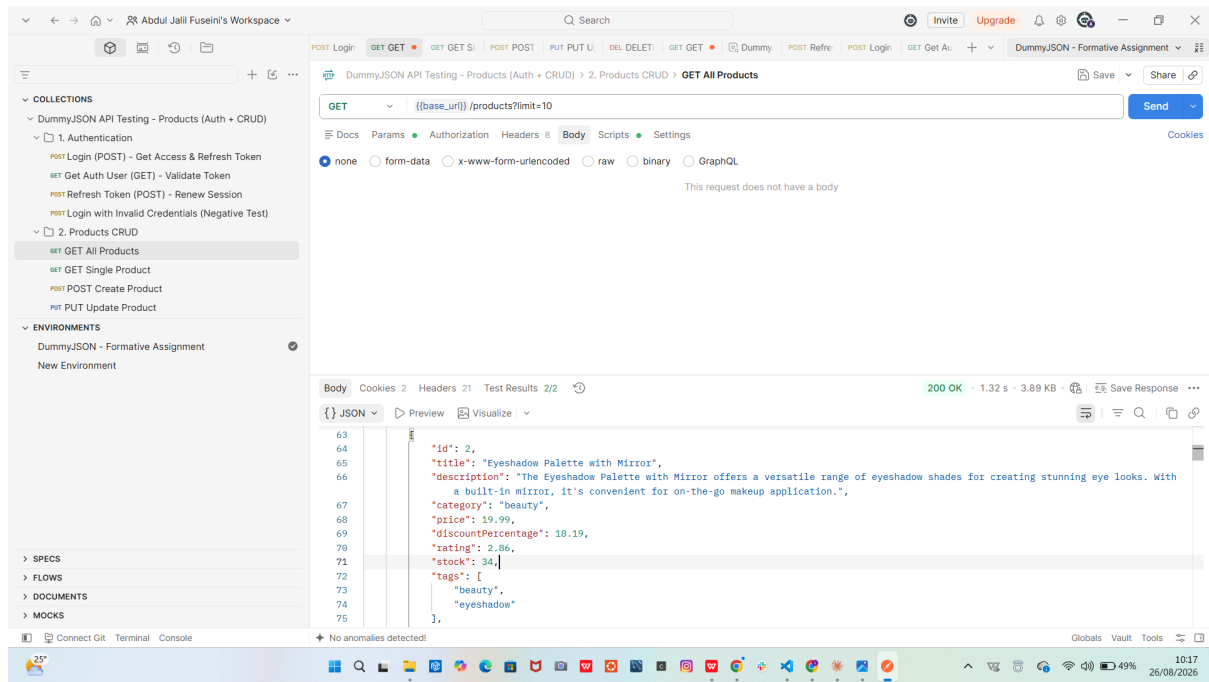
{
  "title": "Cisco Catalyst 9300 Switch",
  "category": "networking-equipment",
  "price": 4599.99,
  "brand": "Cisco",
  "stock": 12
}
```

Response (200 OK) — DummyJSON simulates the insert and returns the object with a newly assigned id (e.g. 195). This id is stored in the environment as `created_product_id`.

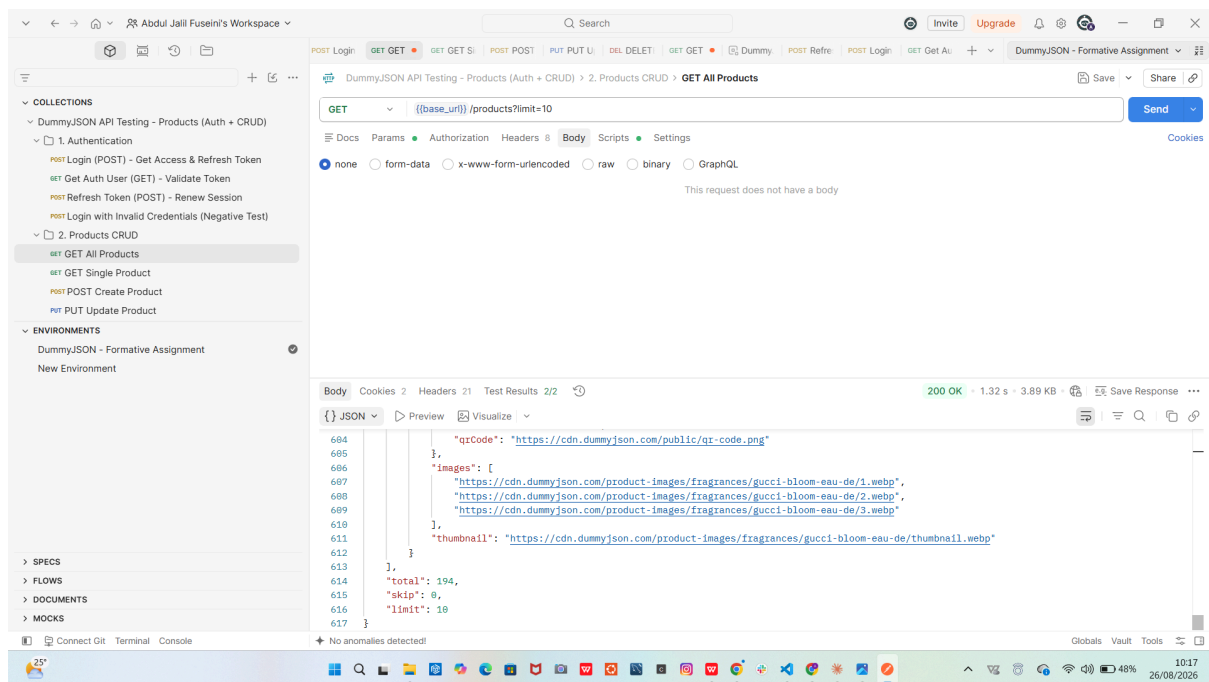


- **READ — GET /products (Retrieving all products)**

GET /products returns a paginated list wrapped in a products array along with total, skip and limit fields. Only the first ten products are returned. The first product's id is automatically captured into the `product_id` environment variable and reused by every subsequent request that needs an ID, so nothing is hardcoded.



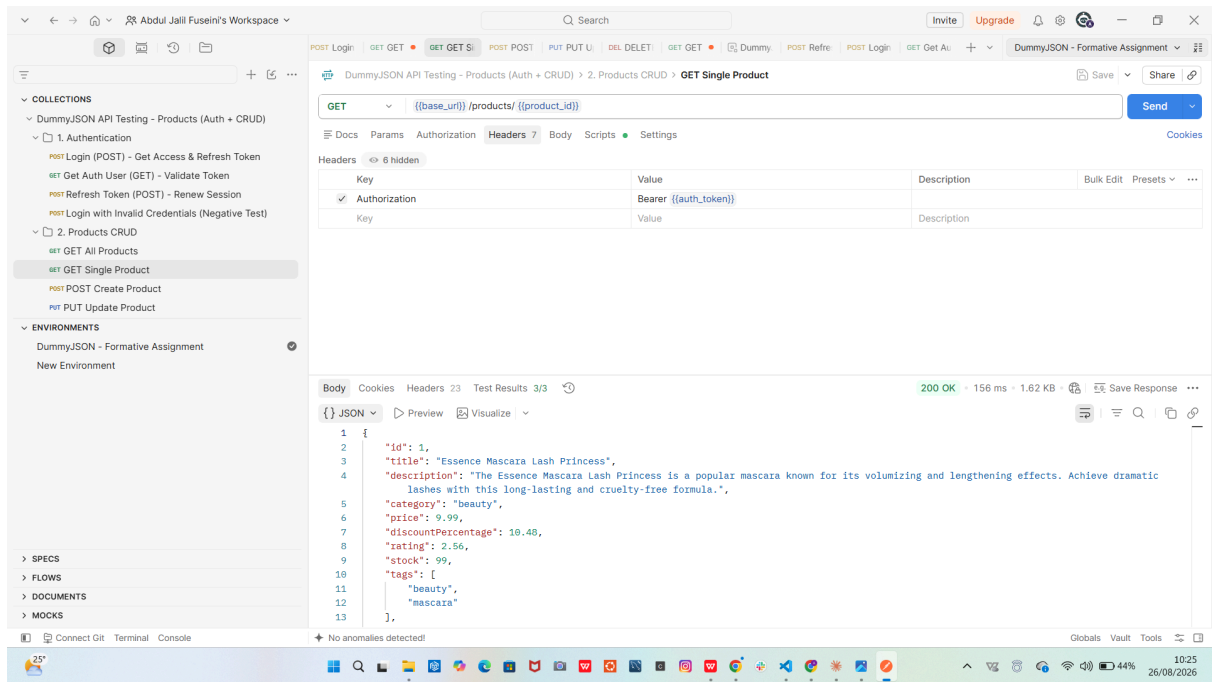
This is just the second product out of the 10 products.



As seen in the screenshot , a total of 194 products are gotten but limited to only 10 .

- **GET /products/{id} (Retrieving only a single product)**

This will return only a single product. By default , when no id is specified , it is bound to use the `product_id` variable stored in the postman environment.



- **UPDATE — PUT /products/{id} (Updating a particular product)**

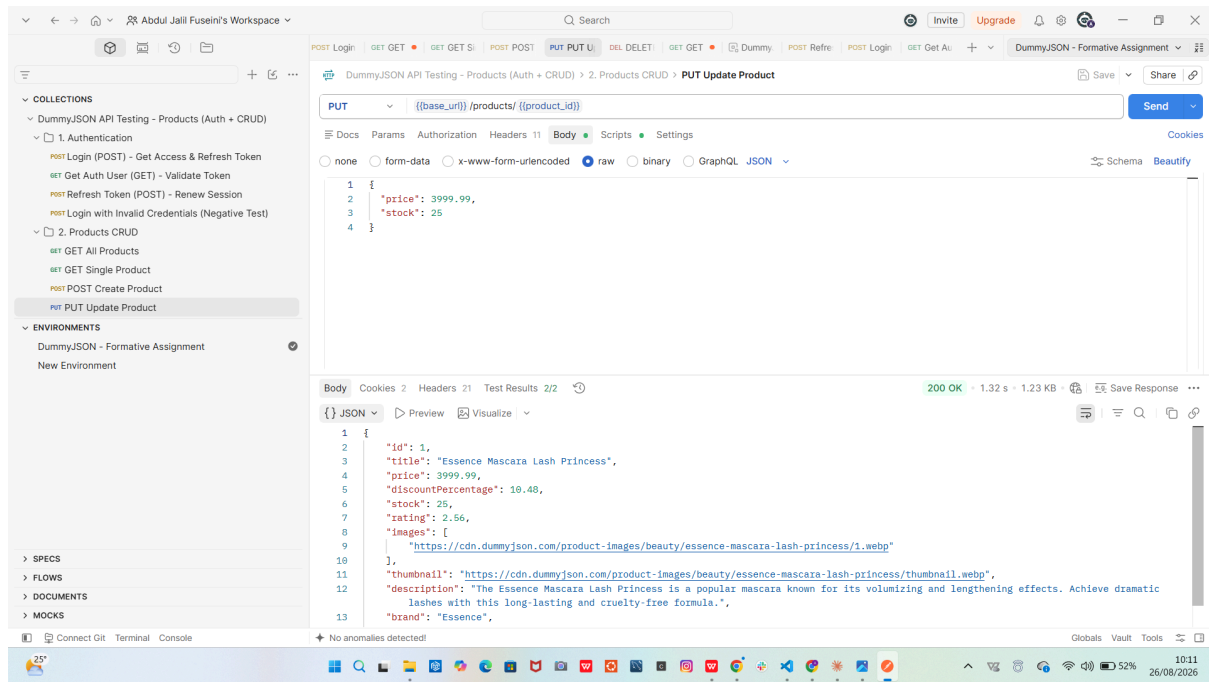
PUT https://dummyjson.com/products/{{product_id}}

Authorization: Bearer {{auth_token}}

Content-Type: application/json

```
{  
  "price": 3999.99,  
  "stock": 25  
}
```

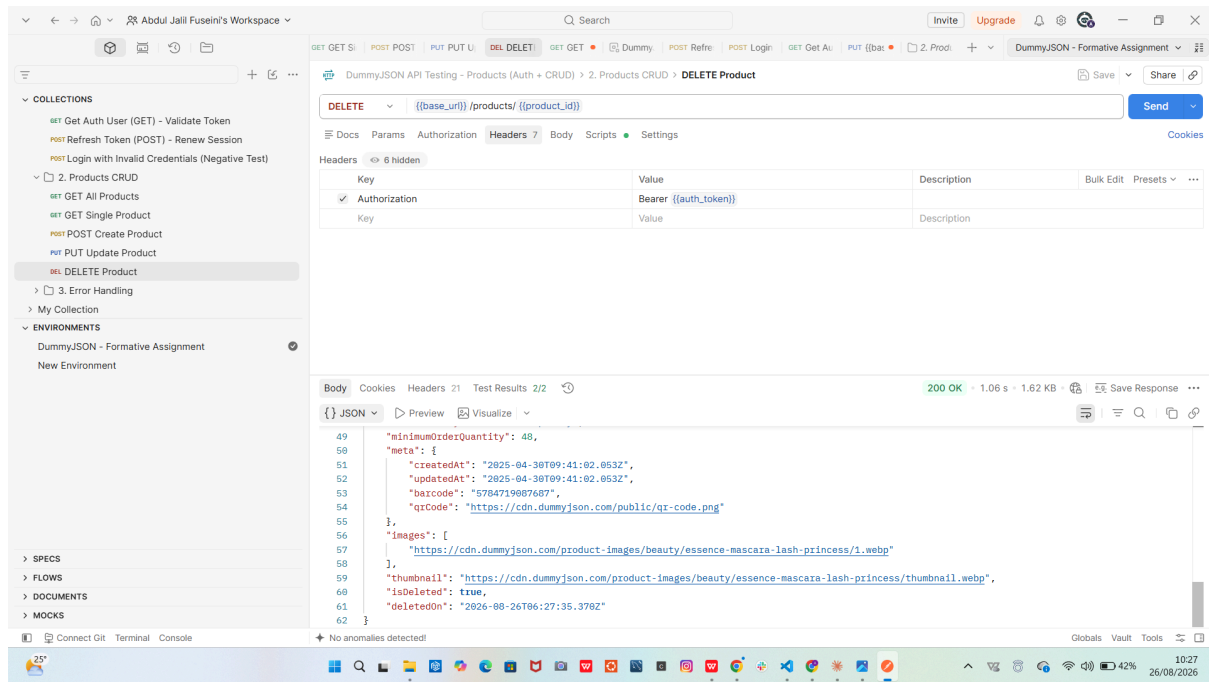
Response (200 OK) returns the product with the submitted fields changed — confirming a partial update rather than a full overwrite since dummyjson is a mock API .Unlike REST API's that will completely overwrite everything(the actual action of put) when the PUT request is made.



Product_id in the postman environment has a value of 1 . And during the request , that value is swapped in to replace the variable.

- **DELETE — DELETE /products/{id}**

Response (200 OK) returns the deleted product with two additional fields: isDeleted: true and deletedOn (an ISO timestamp), which the test script asserts on directly.



This by default will delete the product with the value of the product_id.

5. Postman Environment & Variables

An environment named 'DummyJSON - Formative Assignment' stores every value that would otherwise be hardcoded or re-typed:

```
"id": "e4a1c2d0-9f3b-4e2a-8c1a-dummyjsonenv01",
"name": "DummyJSON - Formative Assignment",
```

- base_url — `https://dummyjson.com`, referenced as `{{base_url}}` in every request URL

```
{
  "key": "base_url",
  "value": "https://dummyjson.com",
  "type": "default",
  "enabled": true
},
```

- username / password — referenced as {{username}} with the value of “emilys” and {{password}} with value “emilyspass” are the login credentials which is kept out of the request body text itself.

```
{  
  
  "key": "username",  
  
  "value": "emilys",  
  
  "type": "default",  
  
  "enabled": true  
  
},  
  
{  
  
  "key": "password",  
  
  "value": "emilyspass",  
  
  "type": "secret",  
  
  "enabled": true  
  
}
```

- auth_token / refresh_token — set as type 'secret' so they are masked in the Postman UI, and populated automatically by scripts rather than typed in. The values for the access and the refresh token is set to empty ,which later on be populated with the result obtained from the test script after the POST {{base_url}}/auth/login request.

```
{  
  
  "key": "auth_token",  
  
  "value": "",  
  
  "type": "secret",  
  
  "enabled": true  
  
},  
  
{
```

```
    "key": "refresh_token",  
  
    "value": "",  
  
    "type": "secret",  
  
    "enabled": true  
  
}
```

- `product_id` — populated dynamically from the GET /products response, then reused in GET-single, PUT, and DELETE requests. It has an initial value of 1.

```
{  
  
    "key": "product_id",  
  
    "value": "1",  
  
    "type": "default",  
  
    "enabled": true  
  
}
```

- `created_product_id` / `logged_in_user_id` — captured from POST and login responses respectively, for traceability. Initially, both are set to empty.

```
{  
  
    "key": "logged_in_user_id",  
  
    "value": "",  
  
    "type": "default",  
  
    "enabled": true  
  
},  
  
{  
  
    "key": "created_product_id",
```

```
"value": "",  
  
"type": "default",  
  
"enabled": true  
  
}
```

Using `{{variable}}` syntax throughout means the entire collection can be pointed at a different account, or re-run after a token expires, without editing a single request.

6. Automation: Pre-request Script & Auto Token Refresh

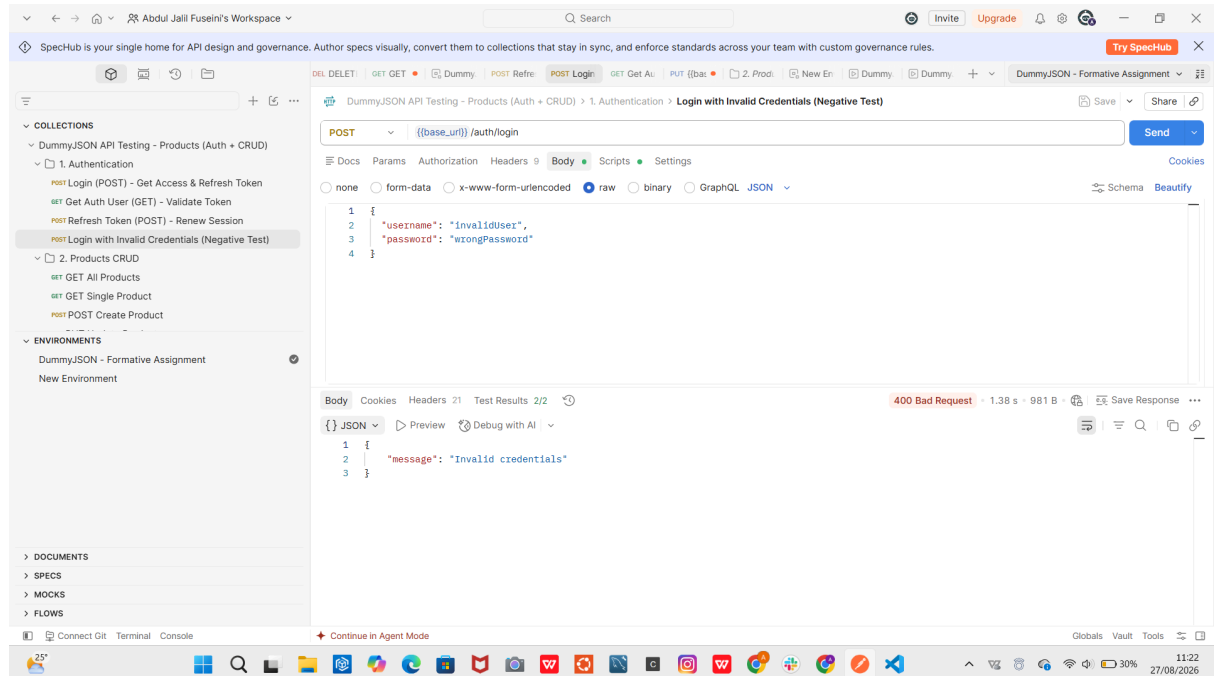
- A collection-level pre-request script (runs before every request in the collection) checks whether `auth_token` is already set in the environment. If it is empty, it calls `pm.sendRequest()` to POST `/auth/login` in the background and stores the returned `accessToken` and `refreshToken` before the actual request executes — directly implementing the pattern shown in the assignment brief, adapted to DummyJSON's response shape (`accessToken/refreshToken` instead of a single token field).

```
const baseUrl = pm.environment.get('base_url');  
const existingToken = pm.environment.get('auth_token');  
  
if (!existingToken) {  
  pm.sendRequest({  
    url: baseUrl + '/auth/login',  
    method: 'POST',  
    header: { 'Content-Type': 'application/json' },  
    body: {  
      mode: 'raw',  
      raw: JSON.stringify({  
        username: pm.environment.get('username'),  
        password: pm.environment.get('password'),  
        expiresInMins: 30  
      })  
    }  
  }, function (err, res) {  
    var jsonData = res.json();  
    pm.environment.set('auth_token', jsonData.accessToken);  
    pm.environment.set('refresh_token', jsonData.refreshToken);  
  });  
}
```

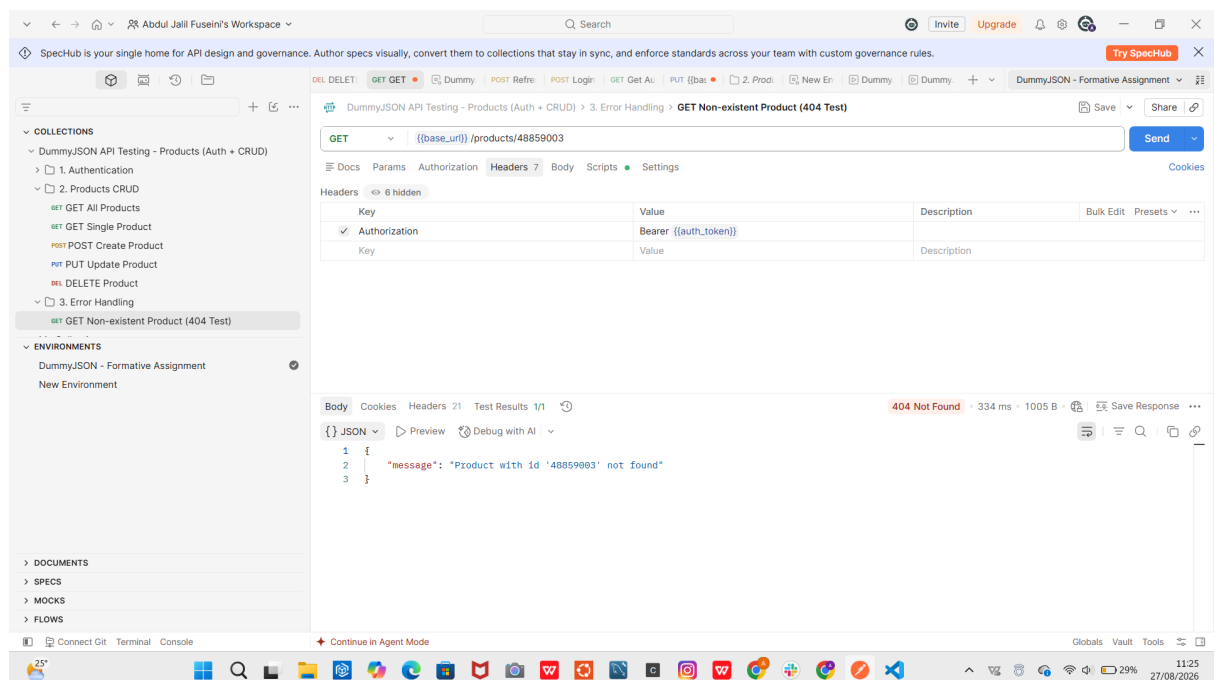
- A dedicated Refresh Token request (POST `/auth/refresh`) additionally demonstrates renewing the session using only the stored `refresh_token`, simulating what would happen automatically if a long-running collection run hit an expired access token.

The collection also includes two negative/error-handling tests, per the assignment tips:

- **Login with Invalid Credentials** — asserts the API returns 400/401 and an error message, rather than succeeding. With this, an invalid username and password not registered with the DummyJson is used.



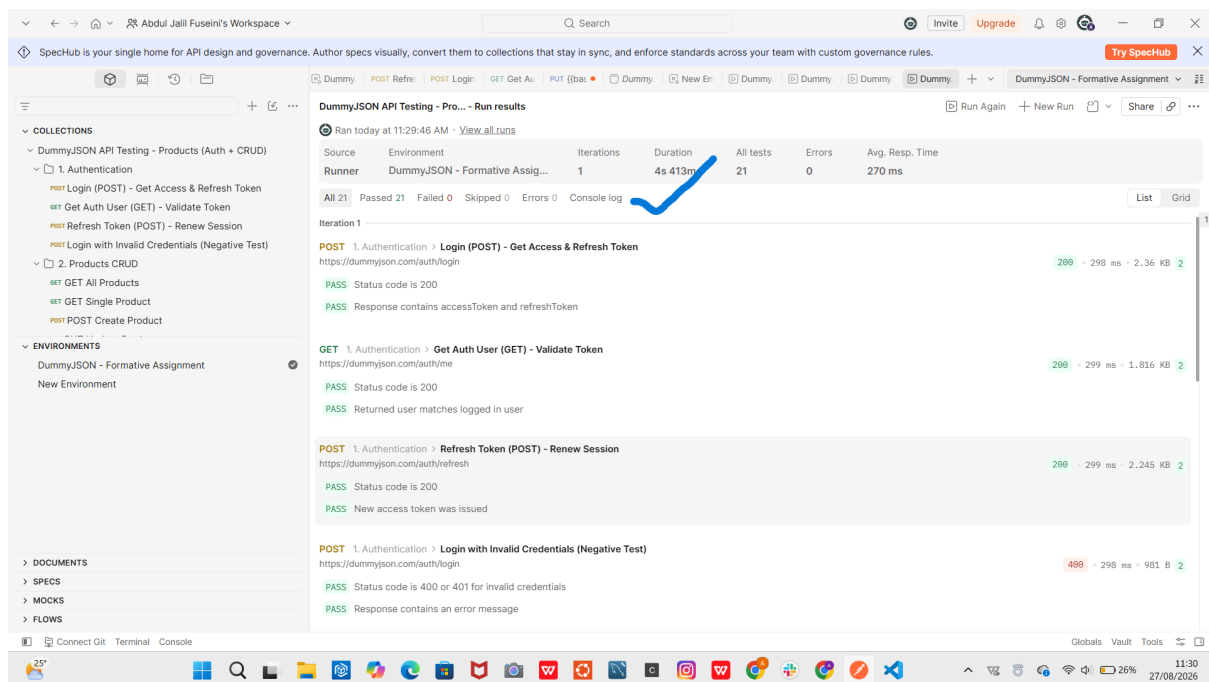
- **GET Non-existent Product** — asserts a 404 is returned for /products/999999.



8. Summary

This exercise covered the full brief: endpoint discovery and documentation, JWT-based authentication with automatic token capture, all four CRUD operations against a live resource, a reusable Postman environment driven entirely by variables, a pre-request script that auto-authenticates, refreshes an expired token and also a test-script-driven automation runnable through the Collection Runner.

This is the summary of the collection runner screen in Postman showing all requests passed and their status (whether passed or failed)



As seen in my screenshot ,all the 21 requests succeeded. Each request has produced the expected results.

Links to my video walkthrough :

https://drive.google.com/file/d/1jZh32dseldpKpm2dmHAs1Pz5qAQd2K4_/view?usp=sharing